Министерство науки и высшего образования Российской Федерации

ФГБОУ ВО «Кубанский государственный технологический университет»

(ФГБОУ ВО «КубГТУ»)

Институт компьютерных систем и информационной безопасности

Кафедра информационных систем и программирования

Направление подготовки 09.03.04 Программная инженерия

Профиль Проектирование и разработка программного обеспечения

КУРСОВАЯ РАБОТА

по дисциплине программирование
(наименование дисциплины)

на тему: Решение типовых задач основных разделов программирования. Вариант 12
(тема курсовой работы)

Выполнила студентка Лисненко Мария Максимовна курса 1 группы 24-КБ-ПР2
(фамилия, имя, отчество)

Допущена к защите _____
(дата)

Руководитель (нормоконтролер) работы _____ к.т.н., доц. В.А. Мурлина
(подпись, дата)

Защищена 12.12.2024г. Оценка отлично
(дата)

Члены комиссии: _____ ассистент П.А. Косолапов

_____ ст. преп. А.А. Ковтун

_____

_____
(подпись, дата)

Краснодар
2024

ФГБОУ ВО «Кубанский государственный технологический университет»
(ФГБОУ ВО «КубГТУ»)

Институт компьютерных систем и информационной безопасности
Кафедра информационных систем и программирования
Направление подготовки 09.03.04 Программная инженерия
Профиль Проектирование и разработка программного обеспечения

УТВЕРЖДАЮ
Зав. кафедрой ИСП, к.т.н., доц.
_____ М.В. Янаева
« 16 » ____ сентября ____ 2024 г.

ЗАДАНИЕ
на курсовую работу

Студентке ____Лисненко Марии Максимовне_____ курса 1 группы 24-КБ-ПР2
Тема работы: Решение типовых задач основных разделов программирования.
Вариант 12
(утверждена указанием директора института № 21-Кн от 10.09.2024г.)
План работы:
1. Изучение алгоритмов решения типовых задач.
2. Реализация на языке C#.
3. Оформление пояснительной записки.
Объем работы:
а) пояснительная записка _____35_____ с.
б) иллюстративная часть _____–_____ лист(-ов)

Рекомендуемая литература:
1. Павловская Т.А. C#. Программирование на языке высокого уровня. Учебник для вузов. – СПб.: Питер, 2014. – 432 с: ил.

Срок выполнения: с « 20 » ____сентября____ по « 20 »____декабря____ 2024 г.

Срок защиты: « 23 » ____декабря____ 2024 г.

Дата выдачи задания: « 20 »____сентября____ 2024 г.

Дата сдачи работы на кафедру: « 30 »____декабря____ 2024 г.

Руководитель работы _____ к.т.н., доц. В.А. Мурлина
(подпись)
Задание приняла студентка _____ М.М. Лисненко
(подпись)

Реферат

Пояснительная записка к курсовой работе содержит: 35 страниц, 12 рисунков, 3 источника информации, 1 приложение.

VISUAL STUDIO 2022, C#, ПРОГРАММЫ, ОДНОМЕРНЫЕ МАССИВЫ, ДВУМЕРНЫЕ МАССИВЫ.

Каждому программисту рано или поздно придется столкнуться с таким понятием, как массив. Практически в любом языке программирования данное понятие встречается. Первое, что нужно понимать, что массив — это всего лишь один из типов данных, который может обрабатывать язык программирования. Как известно, язык программирования может работать с числами, строками и т.д., точно также язык программирования работает с особым типом данных, который называется массив. Массив можно назвать особым типом данных, так как функция хранения других данных в массиве заключается в его структурированном виде. То есть массив можно представить как некое многоуровневое хранилище для типов данных. В массиве могут храниться как строки, числа так и другие типы данных.

Рассмотрим пример массива в жизни для лучшего понимания. Например, гардероб. Представим себе театр, школу и т.д., где есть возможность повесить одежду на крючок с определенным номерком. Подумаем над тем, какими особенностями обладает такой гардероб. Количество одежды, которой в нем можно повесить, строго регламентировано. Есть определенное кол-во крючков (к примеру, 200). Естественно, в реальной жизни можно повесить на 1 крючок несколько элементов одежды, но давайте условимся, что в программирование так делать нельзя, и будет рассматривать, что один крючок предназначен только для одного человека. Помимо этого, есть еще условия, при которых работает данный гардероб: есть возможность по номеру объекта найти этот объект. Заключительное, на каждом крючке будет находиться только одежда и

ничего другого. Вот эти основные параметры характеризуют массив: мы заранее знаем сколько объектов нем хранится, заранее знаем, какие объекты в нем хранятся, и мы можем получить интересующий нас объект по его номеру. Упрощено можно считать, что такая структура данных будет являться массивом.

Теперь, когда известны основные характеристики массива, можно рассмотреть, где начинается массив. Если говорить про гардероб, то нужно бы знать, откуда идет отсчет крючков. Если же рассматривать массив в программировании нужно знать, где находится самый первый объект. Далее нужно знать или задать размер каждого элемента массива. Учитывая, что в данной структуре хранятся все объекты одинакового типа, то заранее известно, сколько места занимает тот или иной объект. Помимо этого, хорошо бы знать количество элементов в массиве, чтобы понимать, что там находится только определенное количество объектов.

Цель данной работы заключается в эффективном применении полученных знаний в области программирования для решения типовых задач основных разделов программирования, а также в понимании принципов их функционирования и методов решения задач, связанных с одномерными и двумерными массивами. Данная курсовая работа важна для развития практических навыков, необходимых для успешной работы с данными структурами, что позволит более уверенно ориентироваться в мире программирования и науке информатика.

Объектом курсовой работы является реализация одномерных и двумерных массивов в языке программирования C#.

Предметом курсовой работы являются алгоритмы обработки массивов в контексте программирования, что позволяет понять их применение и эффективность.

Задачами же являются основные действия с массивами, а именно ввод элементов, сортировка, поиск конкретных элементов и вывод, что позволит лучше понять принципы взаимодействия с набором элементов.

Содержание

Введение 6
1 Нормативные ссылки 8
2 Одномерные массивы 9
2.1 Общие сведения 9
2.2 Постановка задачи 10
2.3 Алгоритм решения задач 11
2.4 Графическое представление решений в виде блок схем 12
2.5 Программная реализация 16
2.6 Результат работы программ 19
3 Двумерные массивы 20
3.1 Общая информация 20
3.2 Постановка задачи 21
3.3 Алгоритм решения задач 22
3.4 Графическое представление решений в виде блок схем 23
3.5 Программная реализация 28
3.6 Результат работы программ 31
Заключение 34
Список используемых источников 35
Приложение А 36

Введение

Актуальность темы:

Изучение программирования в университете является важным этапом подготовки специалистов в области информационных технологий. Применение теоретических знаний на практике позволяет лучше понять основные концепции и методы программирования.

Цель работы:

Целью данной курсовой работы является применение знаний, полученных в процессе обучения, для решения и понимания типовых задач на тему одномерные и двумерные массивы в среде программирования на языке программирования C#. Это включает в себя понимание функционирования одномерных и двумерных массивов, а также разработку алгоритмов для их обработки.

Задачи:

– Получить список заданий.

– Составить алгоритм решения задач и математическое обоснование: для каждой задачи необходимо разработать алгоритм решения и обосновать его с точки зрения математической логики и вычислительной сложности.

– Реализовать решение на C#: написать код для каждой задачи в среде Microsoft Visual Studio, протестировать его и убедиться в корректности выполнения.

– Построить блок-схемы: для визуализации алгоритмов необходимо создать блок-схемы, которые помогут лучше понять логику решения задач.

– Оформить пояснительную записку: собрать все материалы, включая - описания задач, алгоритмы, код и блок-схемы, в единую пояснительную записку.

– Сделать вывод по работе: подвести итоги проделанной работы, оценить достигнутые результаты и выявить возможные направления, где данная тема также пригодиться и будет в дальнейшем использоваться.

В заключении важно сказать, что данная курсовая работа необходима не только для закрепления теоретических знаний, но и развития практических навыков программирования. Решение типовых задач поможет лучше понять основные принципы работы с данными и алгоритмами, что является ключевым аспектом в области программирования.

1 Нормативные ссылки

В данной пояснительной записке использованы ссылки на следующие стандарты:

– ГОСТ 2.105-95 ЕСКД. Общие требования к текстовым документам;

– ГОСТ Р.7.0.5-2008 СИБИД. Библиографическая ссылка. Общие требования и правила составления;

– ГОСТ 7.12-93 СИБИД. Библиографическая ссылка. Сокращение слов на русском языке. Общие требования и правила.

– ГОСТ Р 1.5-2012 Стандартной и Российской Федерации. Стандарты национальные. Правила построения, изложения, оформления и обозначения

– ГОСТ 2.104-2006 ЕСКД. Основные надписи;

2 Одномерные массивы

2.1 Общие сведения

Одномерные массивы – это структуры данных в программировании, представляющие собой упорядоченные коллекции элементов одного типа, расположенных последовательно в памяти компьютера. Эти массивы обладают одним измерением и доступны по индексам, начиная с нуля. Их основная особенность заключается в том, что элементы массива хранятся в памяти последовательно, что обеспечивает быстрый доступ к данным.

Одномерные массивы позволяют эффективно хранить (массивы обеспечивают компактное хранение данных в памяти, так как элементы располагаются последовательно) и обрабатывать данные, так как позволяют компактно организовать информацию одного типа. Они используются для хранения списков, последовательностей чисел, символов, объектов и других типов данных. Преимущество одномерных массивов также заключается в простоте реализации (одномерные массивы легко реализуются и используются в большинстве языков программирования, включая C#, это делает их идеальным выбором для начинающих программистов). Массивы позволяют легко выполнять операции над последовательностями данных, такие как сортировка, поиск и фильтрация.

Примеры применения одномерных массивов:

– Хранение списков (например, массив может использоваться для хранения имена пользователей и их пароли в социальных сетях).

– Работа с последовательностями: Массивы могут быть использованы для реализации алгоритмов сортировки (например, сортировка пузырем) или поиска (например, бинарный поиск).

– Математические вычисления: В научных расчетах массивы часто применяются для хранения векторов и матриц.

– Поиск: реализация алгоритмов поиска, таких как линейный поиск или бинарный поиск для нахождения элементов в массиве.

– Игры (хранение состояния игры, например, список жизней игрока или количество очков).

– Сетевые приложения (хранение адресов IP или других параметров сетевых соединений

Эти примеры демонстрируют гибкость и универсальность одномерных массивов в различных контекстах и приложениях.

Операции, доступные для работы с одномерными массивами, включают инициализацию (задание начальных значений элементов), доступ к элементам по индексу, изменение и чтение значений, а также манипуляции с элементами, такие как сортировка, поиск элемента и другие.

Изучение одномерных массивов в курсовой работе поможет понять основы работы с данными в программировании, их эффективное использование для хранения информации и оптимизацию алгоритмов обработки массивов.

Заключение:

В общем, одномерные массивы представляют удобный и эффективный способ организации данных в программировании. Они обеспечивают возможность сохранения и обработки обширных объемов информации и решения множества задач с применением разнообразных операций. Одномерные массивы являются неотъемлемым инструментом в разработке программного обеспечения и создании алгоритмов.

2.2 Постановка задачи

Ниже представлен порядок заданий:

№1. Дан одномерный массив. Найти среднее арифметическое элементов, кратных 3. Создать другой массив, в который поместить элементы,

превышающие вычисленное среднее арифметическое, изменив их знак на противоположный.

№2. Дан одномерный массив целых чисел. Определить число элементов самой длинной упорядоченной по возрастанию цепочки.

2.3 Алгоритм решения задач

План решения задание №1:

– Создать одномерный целочисленный массив “n” и заполнить его значениями.

– Инициализировать переменные.

– Итерироваться по элементам массива от первого до последнего.

– Если элемент кратен 3, то увеличить “sum” на “n[i]” и увеличить “temp” на 1

– Вычислить среднее арифметическое элементов кратных 3, делением “sum” на “temp”.

– Проходить по каждому элементу “n[i]” в исходном массиве.

– Если “n[i]” больше среднего арифметического, то поместить “-n[i]” в массив «result».

– Вывести “result[i]”.

План решения задание №2:

– Создать одномерный целочисленный массив “n” и заполнить его значениями.

– Инициализировать переменные temp1 и temp2, оба равны 1. 9

– Итерироваться по элементам массива от первого до последнего.

– Проверить, является ли текущий элемент массива больше предыдущего (n[i]> n[i-1]) и, если это так, увеличить temp1 на 1, затем приравнять temp2 к temp1. Иначе, обнулить temp1.

– Повторить для всех элементов массива.

– Вывести temp2.

2.4 Графическое представление решений в виде блок схем.

На рисунке 1 изображена первая часть блок-схемы работы программы №1 и следующие операции, а именно вывод одномерного массива в консоль, проверка деления чисел на 3 без остатка и подсчет данных чисел кратных 3.

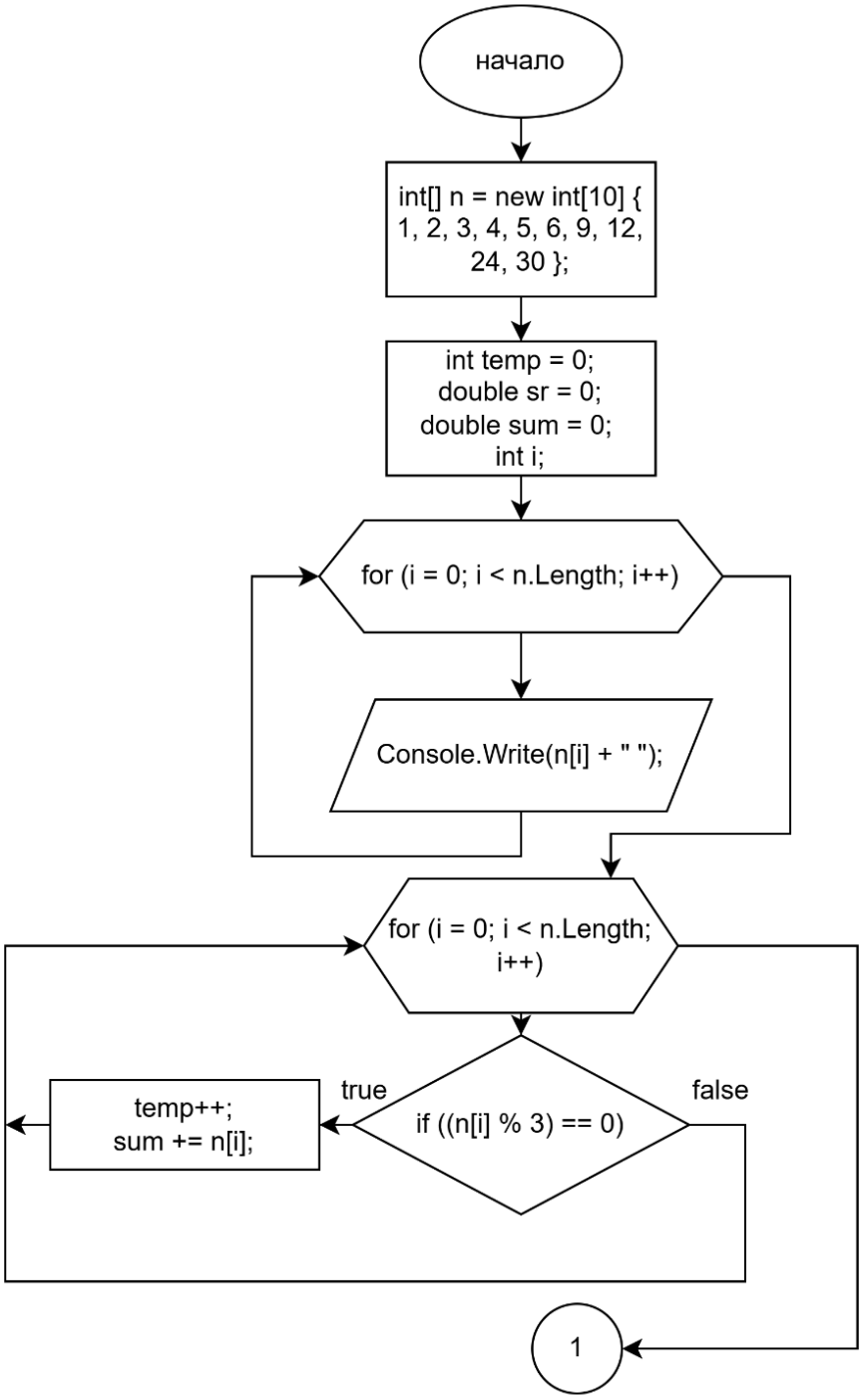


Рисунок 1 – первая часть блок-схемы работы программы №1

На рисунке 2 представлена вторая часть блок-схемы работы программы №1, в которой вычисляется среднее арифметическое элементов, кратных 3, найденных на рис.1, и создания второго массива, в который переходят элементы, превышающие вычисленное среднее арифметическое, изменившие свой знак на противоположный.

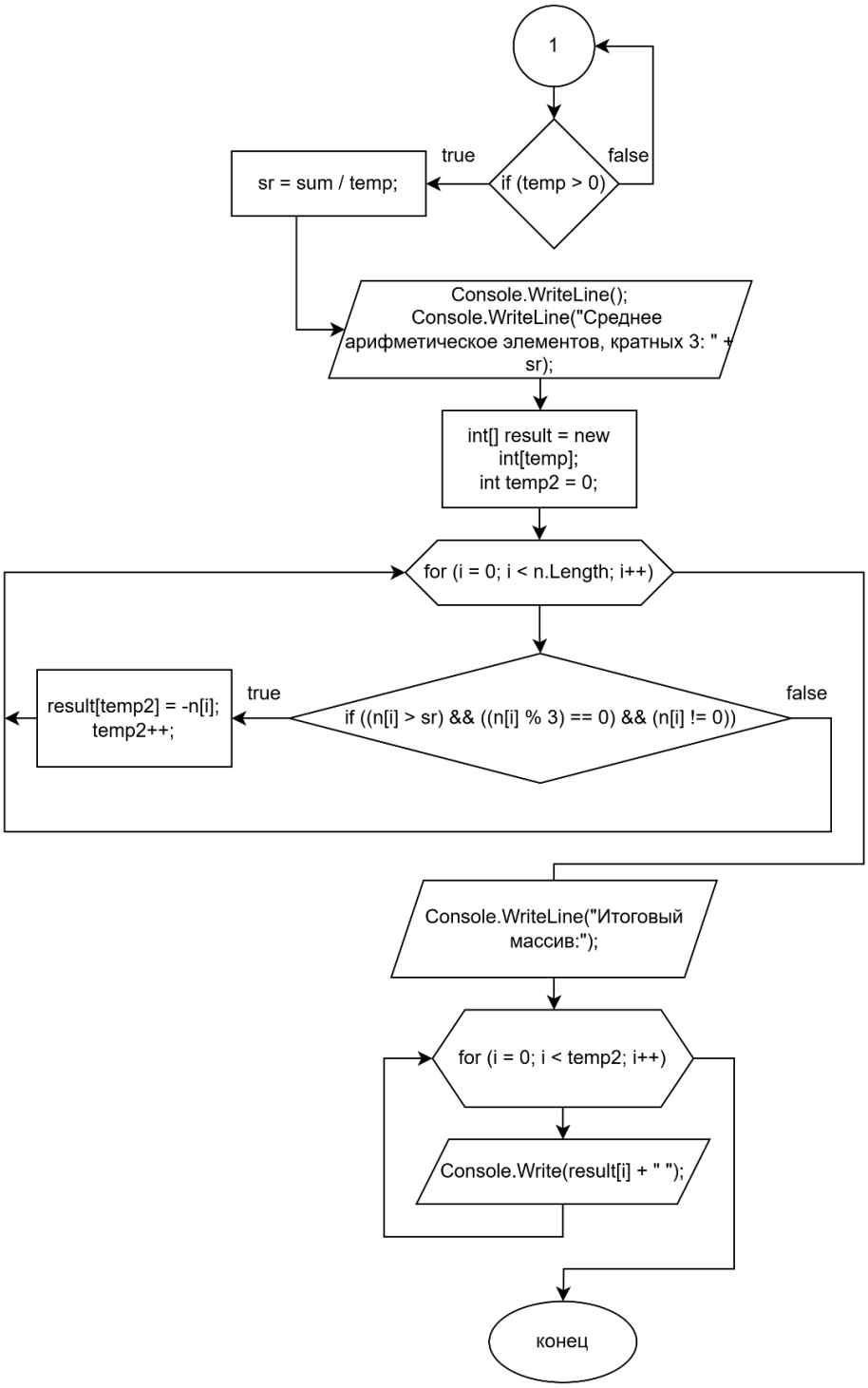


Рисунок 2 – вторая часть блок-схемы работы программы №1

На рисунке 3 изображена первая часть блок-схемы работы программы №2, где изображено создание одномерного массива и заполнение его значениями.

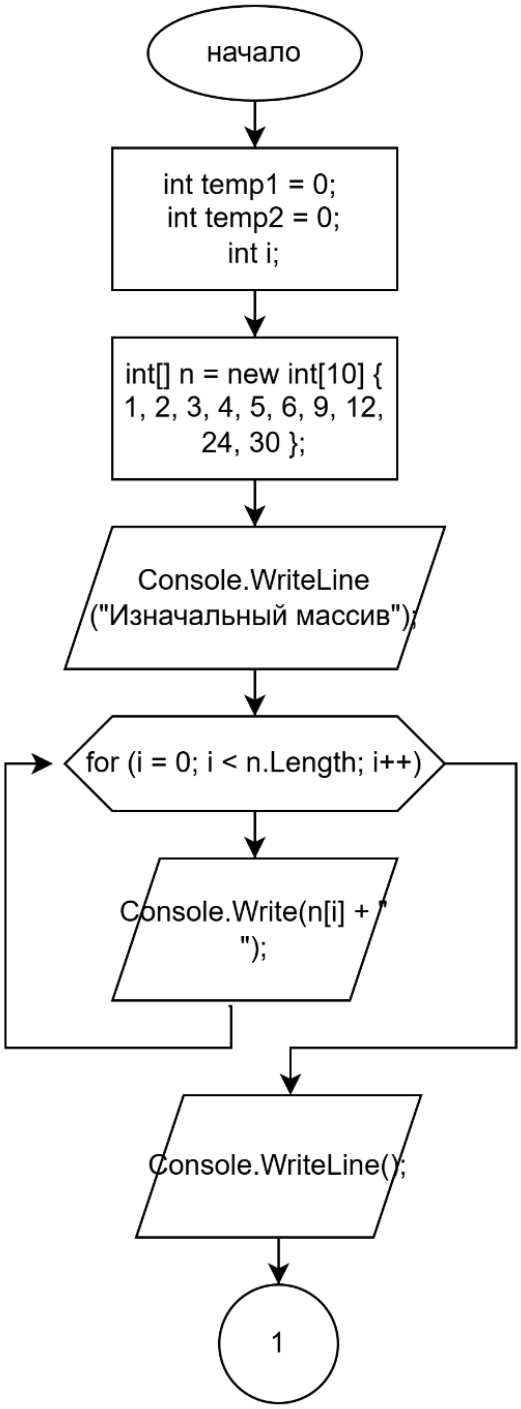


Рисунок 3 – первая часть блок-схемы работы программы №2

На рисунке 4 изображена вторая часть блок-схемы работы программы №2 для определения числа элементов самой длинной упорядоченной по возрастанию цепочки в одномерном массиве.

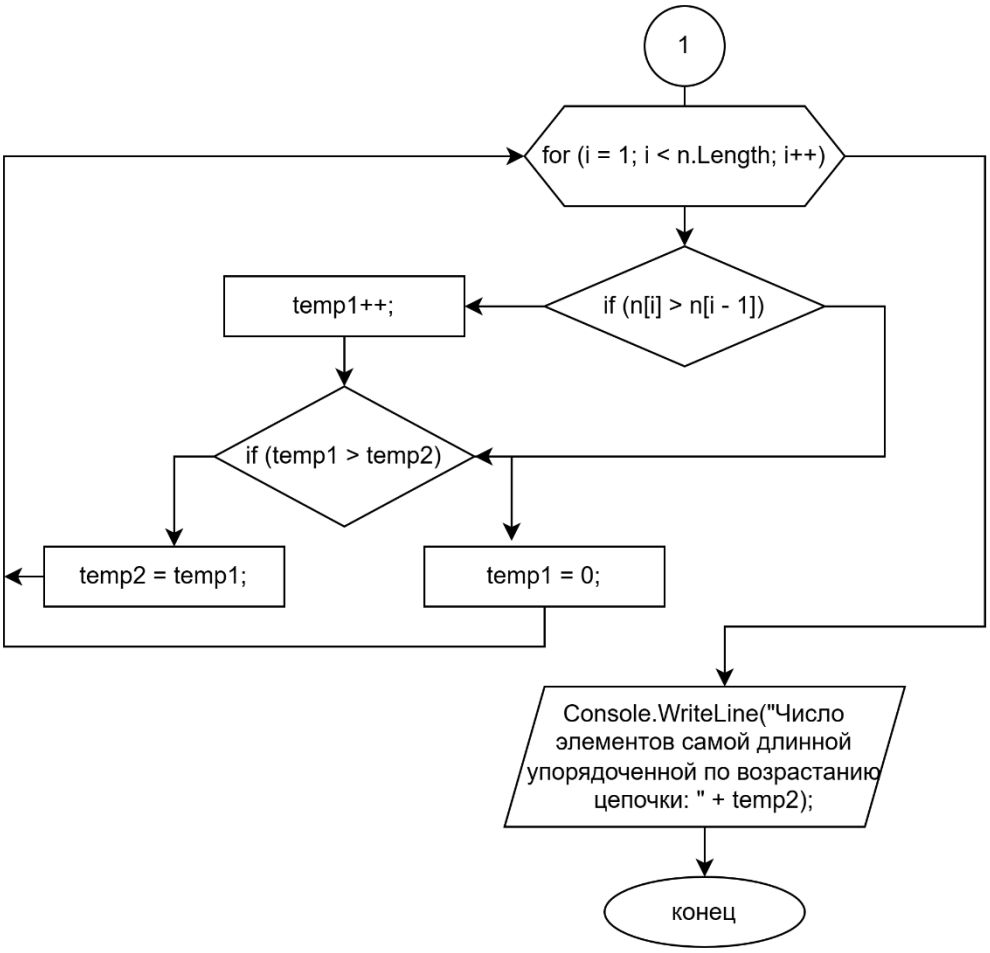


Рисунок 4 – вторая часть блок-схемы работы программы №2

2.5 Программная реализация

Листинг программы №1:

```
using System;
namespace coursework1
{
   internal class Program
   {
      static void Main(string[] args)
      {
         int[] n = new int[10]{ 1, 2, 3, 4, 5, 6, 9, 12, 24, 30 }; //Создание
одномерного целочисленного массив “n” и заполнение его значениями
         int temp = 0;
         double sr = 0;
         double sum = 0; //Инициализирование переменных
         int i;
         Console.WriteLine("Изначальный массив");
```

```
        for (i = 0; i < n.Length; i++) // Прохождение (итерация) по элементам
массива от первого до последнего
        {
            Console.Write(n[i] + " ");
        }
        for (i = 0; i < n.Length; i++)
        {
            if ((n[i] % 3) == 0) // Создание условия, при котором, если элемент
кратен 3, то произойдет увеличение переменной “sum” на “n[i]” и увеличить
переменной “temp” на 1.
            {
                temp++;
                sum += n[i];
            }
        }
        if (temp > 0)
        {
            sr = sum / temp; // Вычисление среднего арифметического
элементов, кратных 3, делением “sum” на “temp”.
        }
        Console.WriteLine();
        Console.WriteLine("Среднее арифметическое элементов, кратных 3: " +
sr);
        int[] result = new int[temp];
        int temp2 = 0;
        for (i = 0; i < n.Length; i++) // Прохождение по каждому элементу “n[i]”
в исходном массиве.
        {
            if ((n[i] > sr) && ((n[i] % 3) == 0) && (n[i] != 0)) // Вычисление
условия, при котором, если “n[i]” больше среднего арифметического, то
поместить “-n[i]” в массив «result»
            {
                result[temp2] = -n[i];
                temp2++;
            }
        }
        Console.WriteLine("Итоговый массив:");
        for (i = 0; i < temp2; i++)
        {
            Console.Write(result[i] + " ");  // Вывод “result[i]”
        }
        Console.WriteLine();
    }
}
```

```
}
```

Листинг программы №2:

```
using System;

namespace coursework2
{
   internal class Program
   {
      static void Main(string[] args)
      {
         int temp1 = 0; // Инициализирование переменных temp1 и temp2
         int temp2 = 0;
         int i;

         int[] n = new int[10] { 1, 2, 3, 4, 5, 6, 9, 12, 24, 30}; // Создание
одномерного целочисленного массива “n” и заполнение его значениями.
         Console.WriteLine("Изначальный массив");
         for (i = 0; i < n.Length; i++) // Повтор для всех элементов массива
         {
            Console.Write(n[i] + " ");
         }
         Console.WriteLine();

         for (i = 1; i < n.Length; i++) // Прохождение (итерация) по элементам
массива от первого до последнего.
         {
            if (n[i] > n[i - 1]) //Проверка, является ли текущий элемент массива
больше предыдущего (n[i] > n[i-1]) и, если это так, происходит увеличение
temp1 на 1, затем приравнивание temp2 к temp1.
            {
               temp1++;

               if (temp1 > temp2)
               {
                  temp2 = temp1;
               }
            }
            else
            {
               temp1 = 0; // Иначе, обнуление переменной temp1.
            }
         }
```

```
        Console.WriteLine("Число элементов самой длинной упорядоченной по возрастанию цепочки: " +temp2);
        }
    }
}
```

2.6 Результат работы программ

На иллюстрации 4 представлен итог работы программы №1. После завершения выполнения программы в консоли будет показан исходный массив, среднее значение элементов, кратных 3, а также итоговый массив.

```
Изначальный массив
1 2 3 4 5 6 9 12 24 30
Среднее арифметическое элементов, кратных 3: 14
Итоговый массив:
-24 -30
```

Рисунок 5 – результат работы программы №1

На рисунке 5 изображен итог работы программы №2. После выполнения программы в консоли будет показан исходный массив, а также число элементов самой длинной упорядоченной по возрастанию цепочки.

```
Изначальный массив
1 2 3 4 5 6 9 12 24 30
Число элементов самой длинной упорядоченной по возрастанию цепочки: 9
```

Рисунок 6 – результат работы программы №2

3 Двумерные массивы

3.1 Общая информация

Двумерный массив представляет собой массив массивов, то есть структуру данных, в которой каждый элемент массива является также массивом. Он используется для хранения данных в виде таблицы, где данные организованы в виде строк и столбцов.

В отличие от одномерных массивов, где доступ к элементам осуществляется по одному индексу, в двумерных массивах для доступа к элементу используется два индекса: один для указания строки, второй для указания столбца.

Двумерные массивы широко используются для представления матриц, изображений, таблиц и других структур данных, где данные организованы в виде двухмерной структуры. Они позволяют эффективно обрабатывать данные, проводить операции над матрицами, выполнять поиск, сортировку и другие манипуляции с данными в удобной форме.

Как правило, при решении задач с двумерными массивами используется два вложенных друг в друга цикла for, которые перебирают значения массива: внешний цикл перебирает строки, внутренний – столбцы.

Примеры применения одномерных массивов:

– Структурирование данных: двумерные массивы позволяют удобно хранить и управлять данными, которые естественным образом имеют две измерения, например, координаты (x, y), таблицы значений или изображения.

– Математические операции: они широко используются в линейной алгебре для выполнения операций с матрицами, таких как сложение, умножение и транспонирование.

– Графика и визуализация: в компьютерной графике двумерные массивы используются для представления пикселей изображений, где каждый элемент массива соответствует цвету или яркости пикселя.

– Игровая разработка: в играх двумерные массивы могут представлять игровые поля, карты или уровни, где каждый элемент массива отвечает за состояние определенной клетки (например, свободная ячейка, стенка или враг).

– Обработка данных: двумерные массивы позволяют эффективно обрабатывать и анализировать большие объемы данных, такие как таблицы в базах данных или результаты экспериментов.

– Алгоритмы поиска и сортировки: они могут использоваться для реализации различных алгоритмов, включая сортировку и поиск в таблицах.

– Системы координат: двумерные массивы помогают в представлении и манипуляции с геометрическими фигурами и объектами в пространстве.

Эти примеры показывают, как двумерные массивы могут быть полезны в самых разных контекстах.

В заключение, двумерные массивы являются ключевым инструментом для организации, анализа и обработки данных в различных областях, включая программирование, математику, графику и научные исследования. Их использование позволяет значительно упростить решение многих задач и повысить эффективность работы с данными.

3.2 Постановка задачи

Ниже представлен порядок заданий:

№1. Дан двумерный целочисленный массив А(п,т), просматриваемый через окно 3х3. Определить число окон, в которых все элементы положительны.

№2. Результаты сессии, состоящей из трех экзаменов, для группы из п студентов представлены матрицей К(п, 3). Оценка ставится по четырехбалльной системе; неявка обозначена единицей. Подсчитать количество неявок, неудовлетворительных, удовлетворительных, хороших и отличных оценок по каждому экзамену.

3.3 Алгоритм решения задач

План решения задание №3:

– Создать двумерный целочисленный массив array2d размером m x n

– Запросить у пользователя размеры массива m и n и сохранить их в переменные m и n соответственно.

– Заполнить массив array2d случайными числами от -1 до 8.

– Создать переменные num, temp и temp2 и инициализировать их значением 1 и 0.

– Создать двумерный целочисленный массив okno размером 3 x 3.

– Заполнить массив okno значениями из массива array2d, начиная с позиции array2d[i, j].

– Проверить каждый элемент массива okno на отрицательность.

– Если хотя бы один элемент отрицательный, прервать цикл и перейти к следующему окну. Если все элементы положительные, увеличить значение переменной temp на 1.

– После завершения проверки всех элементов массива okno, если значение переменной temp равно 9, увеличить значение переменной temp2 на 1. Затем сбросить значение переменной temp в 0.

– Вывести на экран массив array2d, массив okno а также число окон, в которых все элементы положительны (значение переменной temp2).

План решения задание №4:

– Запросить у пользователя неизвестные данные о студентах и экзаменах:

– Создать двумерный массив grades с размерами [studentCount, examCount], чтобы хранить оценки каждого студента за каждый экзамен.

– Заполнить массив grades оценками оценки должны быть в диапазоне от 1 до 4, где 1 — это неявка.

– Подсчитать количество оценок по каждому экзамену у каждого ученика, с помощью цикла для обхода каждого экзамена, с помощью счетчика и конструкции switch.

– Вывести на экран результат: количество неявок, количество неудовлетворительных оценок, количество удовлетворительных оценок, количество отличных оценок.

3.4 Графическое представление решений в виде блок схем.

На рисунке 7 изображена первая часть блок-схемы работы программы №3, где изображено создание двумерного массива для создания матрицы размером m x n и заполнение его значениями, а также двумерного целочисленного массива размера 3 x 3, далее выполняется такая операция, как проверка элементов массива на отрицательность.

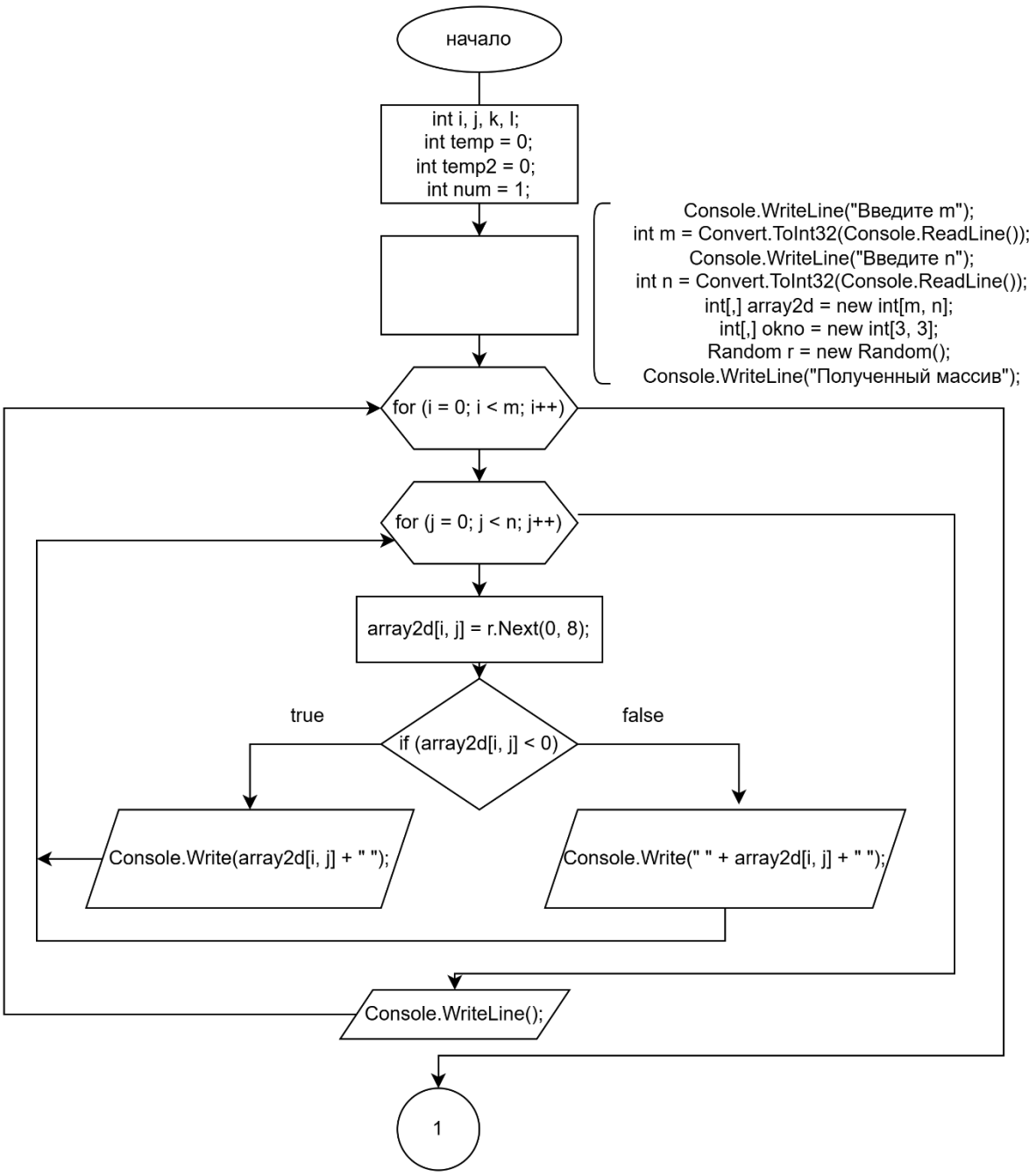


Рисунок 7 – первая часть блок-схемы работы программы №3

На рисунке 8 изображена вторая часть блок-схемы работы программы №3, где изображено продолжение работы циклов поиска положительных элементов и переход к следующим окнам, в случае если число оказалось отрицательным.

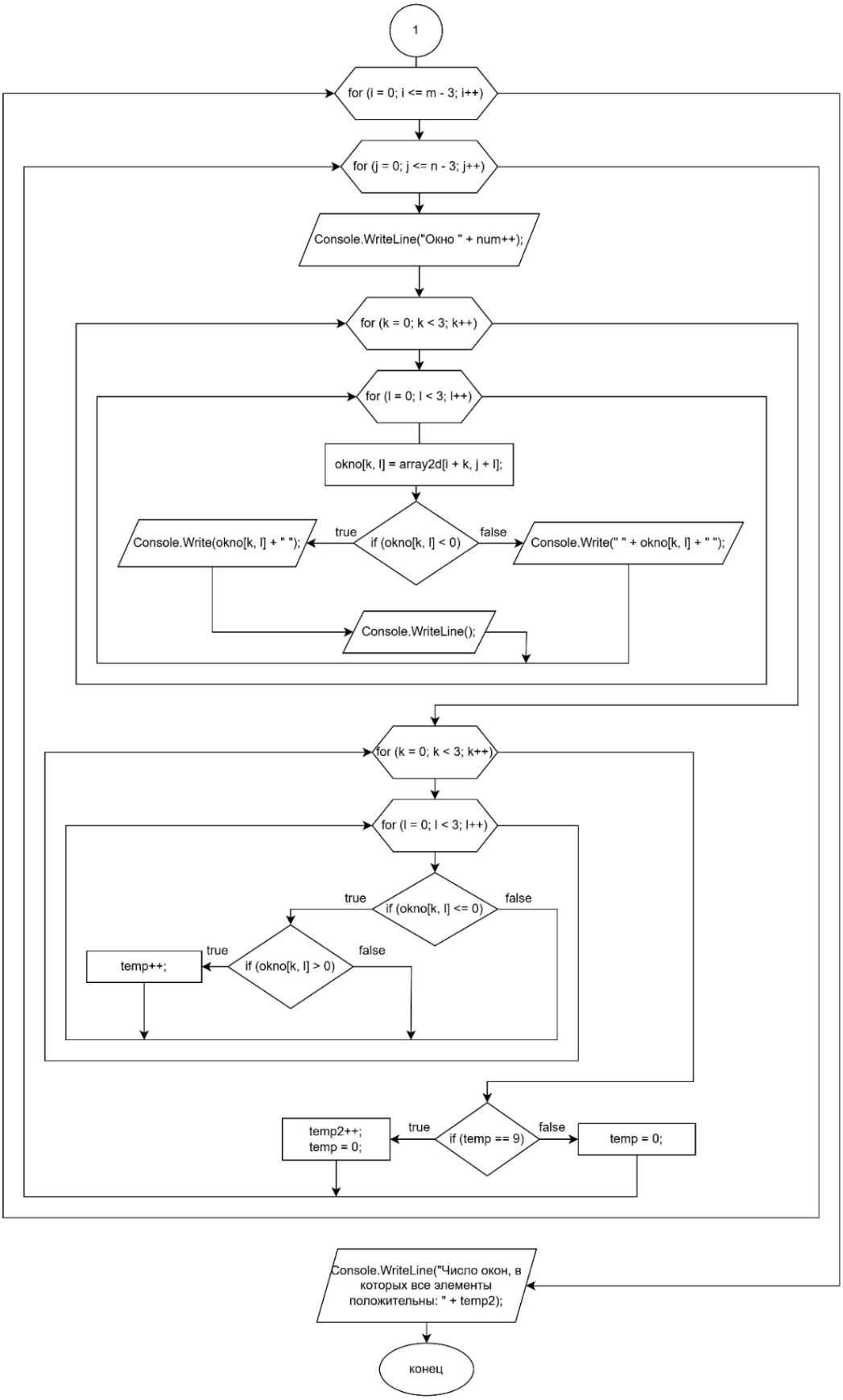


Рисунок 8 – вторая часть блок-схемы работы программы №3

На рисунке 9 изображена первая часть блок-схемы работы программы №4, где изображено создание двумерного массива для хранения оценок и заполнение его значениями.

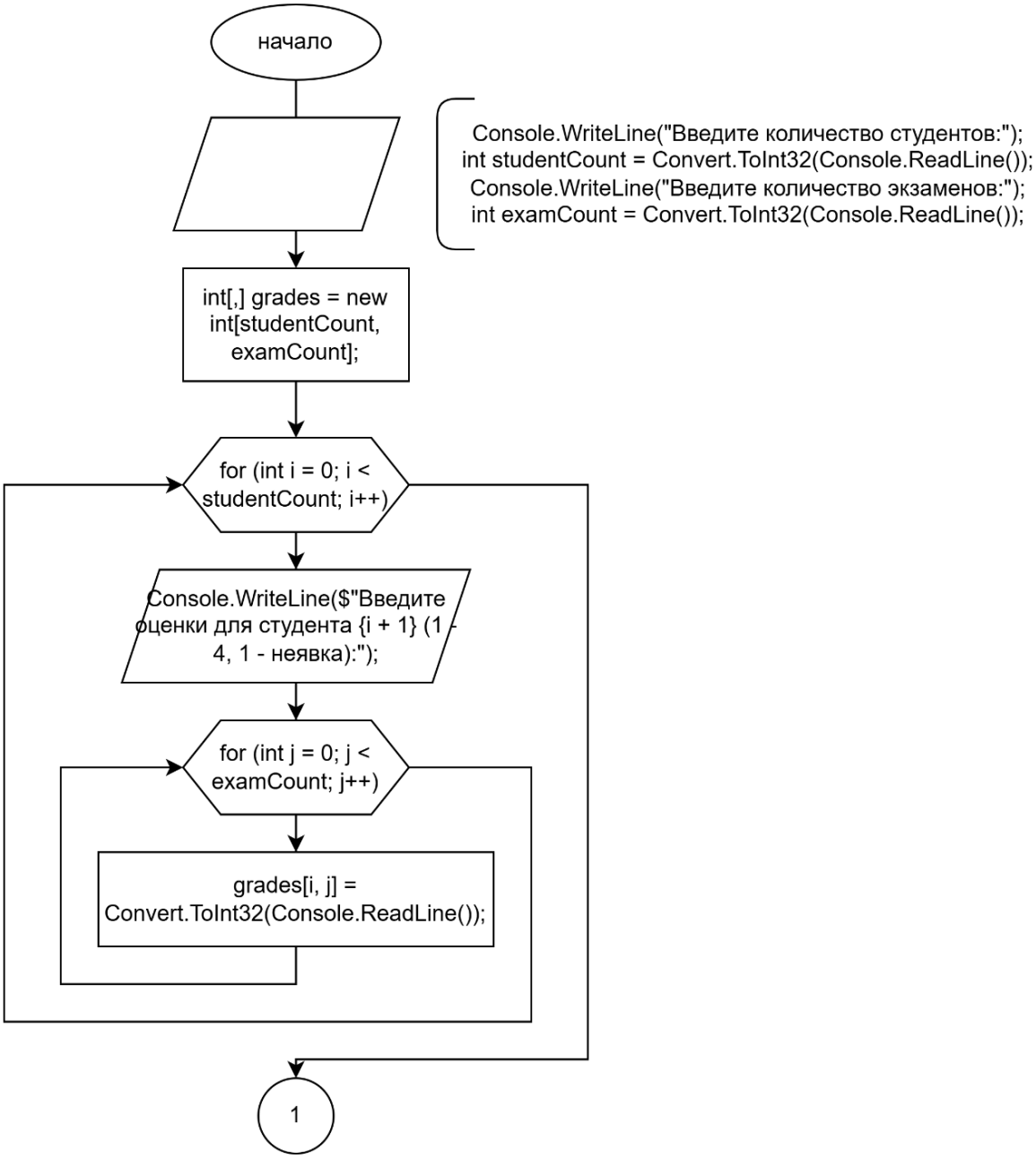


Рисунок 9 – первая часть блок-схемы работы программы №4

На рисунке 10 изображена вторая часть блок-схемы работы программы №4, где изображен подсечет оценок.

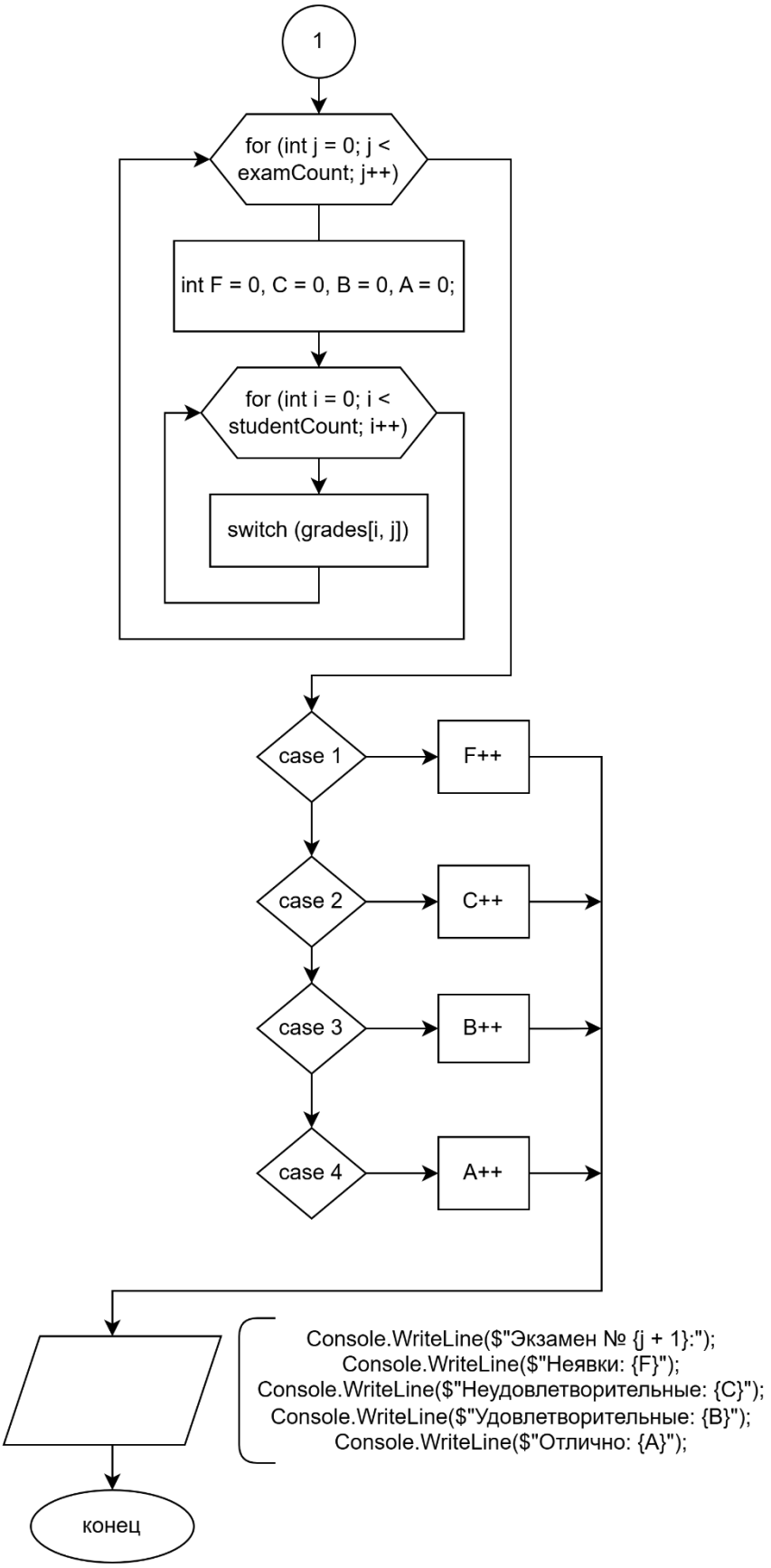


Рисунок 10 – вторая часть блок-схемы работы программы №4

3.5 Программная реализация

Листинг программы №1:

```
using System;

namespace coursework3
{
   internal class Program
   {
      static void Main(string[] args)
      {
         int i, j, k, l; //объявление переменных (переменных)
         int temp = 0; // объявление счетчика положительных элементов
         int temp2 = 0; // объявление счетчика окон
         int num = 1; // объявлнение нумерации окон
         Console.WriteLine("Введите m"); //ввод размеров массивов (кол-во
строк и столбцов)
         int m = Convert.ToInt32(Console.ReadLine());
         Console.WriteLine("Введите n");
         int n = Convert.ToInt32(Console.ReadLine());
         int[,] array2d = new int[m, n]; // создание двух двумерных массивов (для
случайных чисел)
         int[,] okno = new int[3, 3]; // для заданного  окна
         Random r = new Random(); //создание рандома
         Console.WriteLine("Полученный массив");
         for (i = 0; i < m; i++)
         {
            for (j = 0; j < n; j++)
            {
               array2d[i, j] = r.Next(0,8);
               if (array2d[i, j] < 0)
               {
                  Console.Write(array2d[i, j] + " ");
               }
               else
               {
                  Console.Write(" " + array2d[i, j] + " ");
               }
            }
            Console.WriteLine();
         }
         for (i = 0; i <= m - 3; i++) //поиск окон 3*3
         {
```

```
for (j = 0; j <= n - 3; j++)
{
    Console.WriteLine("Окно " + num++);
    for (k = 0; k < 3; k++)
    {
        for (l = 0; l < 3; l++)
        {
            okno[k, l] = array2d[i + k, j + l];



            if (okno[k, l] < 0) //проверка элементов для подбора
            {
                Console.Write(okno[k, l] + " ");
            }
            else
            {
                Console.Write(" " + okno[k, l] + " ");
            }
        }
        Console.WriteLine();
    }

    for (k = 0; k < 3; k++)
    {
        for (l = 0; l < 3; l++)
        {
            if (okno[k, l] <= 0)
            {
                break;
            }
            if (okno[k, l] > 0)
            {
                temp++;
            }

        }
    }
    if (temp == 9) //проверка на положительный знак
    {
        temp2++;
        temp = 0;
    }
    else
```

```
                {
                    temp = 0;
                }
            }
        }
        Console.WriteLine("Число окон, в которых все элементы положительны: " + temp2);
    }
  }
}
```

Листинг программы №2:

```
using System;

namespace SessionResults
{
  class Program
  {
    static void Main(string[] args)
    {
        Console.WriteLine("Введите количество студентов:");
        int studentCount = Convert.ToInt32(Console.ReadLine());
        Console.WriteLine("Введите количество экзаменов:");
        int examCount = Convert.ToInt32(Console.ReadLine());

        int[,] grades = new int[studentCount, examCount]; // Создание двумерного массива для хранения оценок

        // Заполнение массива оценками
        for (int i = 0; i < studentCount; i++)
        {
            Console.WriteLine($"Введите оценки для студента {i + 1} (1 - 4, 1 - неявка):");
            for (int j = 0; j < examCount; j++)
            {
                grades[i, j] = Convert.ToInt32(Console.ReadLine());
            }
        }

        // Подсчет оценок
        for (int j = 0; j < examCount; j++)
        {
            int F = 0, C = 0, B = 0, A = 0;
```

```
            for (int i = 0; i < studentCount; i++)
            {
                switch (grades[i, j])
                {
                    case 1: F++; break; // Неявка
                    case 2: C++; break; // Неудовлетворительная
                    case 3: B++; break; // Удовлетворительная
                    case 4: A++; break; // Отлично
                }
            }

            // Вывод результатов
            Console.WriteLine($"Экзамен № {j + 1}:");
            Console.WriteLine($"Неявки: {F}");
            Console.WriteLine($"Неудовлетворительные: {C}");
            Console.WriteLine($"Удовлетворительные: {B}");
            Console.WriteLine($"Отлично: {A}");
        }
    }
  }
}
```

3.6 Результат работы программ

На рисунке 11 изображен результат работы программы №3. После выполнения программы в консоли будет отображаться полученным массив, окна 3x3, а также подсчет окон, в которых все элементы положительны.

```
Введите m
4
Введите n
4
Полученный массив
 7  1  1  0
 7  1  3  5
 6  5  4  6
 6  7  7  3
Окно 1
 7  1  1
 7  1  3
 6  5  4
Окно 2
 1  1  0
 1  3  5
 5  4  6
Окно 3
 7  1  3
 6  5  4
 6  7  7
Окно 4
 1  3  5
 5  4  6
 7  7  3
Число окон, в которых все элементы положительны: 3
```

Рисунок 11 – результат работы программы №3

На рисунке 12 изображен итог работы программы №4. После выполнения программы выводиться на консоль результат: количество неявок, количество неудовлетворительных оценок, количество удовлетворительных оценок, количество отличных оценок.

```
Введите количество студентов:
3
Введите количество экзаменов:
3
Введите оценки для студента 1 (1 - 4, 1 - неявка):
4
4
3
Введите оценки для студента 2 (1 - 4, 1 - неявка):
2
1
1
Введите оценки для студента 3 (1 - 4, 1 - неявка):
3
2
4
Экзамен № 1:
Неявки: 0
Неудовлетворительные: 1
Удовлетворительные: 1
Отлично: 1
Экзамен № 2:
Неявки: 1
Неудовлетворительные: 1
Удовлетворительные: 0
Отлично: 1
Экзамен № 3:
Неявки: 1
Неудовлетворительные: 0
Удовлетворительные: 1
Отлично: 1
```

Рисунок 12 – результат работы программы №4

Заключение

В заключении можно добавить, что изучение задач на одномерные и двумерные массивы является важным шагом в освоении программирования и расширении своих навыков. Они открывают множество возможностей для создания более сложных и функциональных программ, а также помогают развивать логическое мышление и алгоритмическое мышление.

В целом, выполненные задания не только подтвердили теоретические знания в области программирования и алгоритмического мышления, но и продемонстрировали практические навыки разработки эффективных решений. Практическое применение этих алгоритмов подчёркивает важность сочетания теории и практики для успешного решения реальных задач в области программирования.

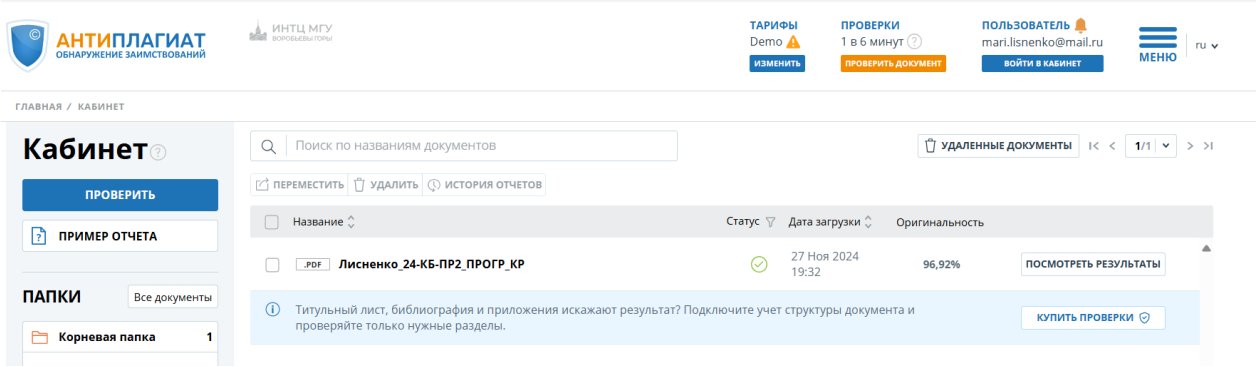
Цель работы достигнута, все предложенные задачи решены, а их решения представлены в виде математического обоснования, блок-схем и листингов программ. Хотя решенные задачи являются типовыми, однако их решения являются хорошим образцом использования характерного синтаксиса и могут войти в сборник задач по программированию на языке C#.

Список используемых источников

1. Павловская Т.А. C#. Программирование на языке высокого уровня: Учебник для вузов. – СПб.: Питер, 2014. – 432 с.: ил.

2. Троелсен Эндрю, Джепикс Филипп Язык программирования C# 7 и платформы .NET и .NET Core, 8-е изд.: Пер. с англ. – СПб.: ООО «Диалектика», 2018 – 1328 с.: ил.

3. Марков В.Н. Алгоритмы и структуры данных: учеб. пособие/ В.Н. Марков. – Краснодар: Изд. ФГБОУ ВО «КубГТУ», 2022. – 207 с.

Приложение А

Проверка на антиплагиат